

Министерство науки и высшего образования Российской Федерации
федеральное государственное автономное образовательное учреждение
высшего образования

«Национальный исследовательский университет ИТМО»

Факультет прикладной информатики

Отчёт по курсовой работе
дисциплины Мобильная разработка (Android и IOs) на тему
“Разработка Android-приложения Мессенджер”

Выполнила:

студентка группы К3444
поток МОБРАЗ 2.1
Самойленко Ева Игоревна

Проверил:

преподаватель ФПИН ИТМО
Шатинский Григорий Сергеевич

Санкт-Петербург, 2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	3
1 Структура проекта.....	5
2 Создание скелета приложения с помощью Activity и Fragment.....	8
2.1 Фрагмент ленты (NewsFragment).....	10
2.2 Фрагмент профиля (ProfileFragment).....	11
2.3 Фрагмент настроек (SettingsFragment).....	11
3 Внедрение ViewModel.....	12
3.1 ProfileViewModel.....	12
3.2 Изменение фрагмента Профиля.....	14
3.3 SettingsViewModel.....	14
4 Загрузка данных из API.....	16
5 Отображение списка на странице Новостей.....	20
6 Улучшение пользовательского интерфейса.....	24
ЗАКЛЮЧЕНИЕ.....	26

ВВЕДЕНИЕ

В рамках курсового проекта за семестр была выполнена разработка Android-приложения «Мессенджер» с использованием современных инструментов и архитектурных подходов, обеспечивающих стабильность, производительность и удобство использования. Проект реализован поэтапно через четыре лабораторные работы, каждая из которых направлена на освоение ключевых аспектов создания мобильных приложений под платформу Android.

Первая лабораторная работа была посвящена основам построения пользовательского интерфейса и навигации в приложении. Была разработана базовая структура с главной активностью, фрагментами и навигацией через Bottom Navigation, что позволило создать понятный и интуитивно понятный интерфейс для перехода между экранами ленты, профиля и настроек. Особое внимание уделялось соблюдению жизненного цикла компонентов и использованию стандартных средств Android Navigation.

Во второй работе основное внимание уделялось архитектурному паттерну MVVM (Model-View-ViewModel) и управлению состоянием приложения. Были реализованы ViewModel для хранения данных профиля и настроек, обеспечено реактивное обновление интерфейса с помощью LiveData, а также сохранение состояния при повороте экрана. Это позволило отделить логику приложения от пользовательского интерфейса и повысить его устойчивость к изменениям.

Третья лабораторная работа была направлена на интеграцию приложения с сетевыми сервисами и локальным хранилищем данных. С помощью Retrofit и Kotlin Coroutines была организована загрузка данных из публичного API, а также их сохранение в локальную базу данных Room. Это обеспечило возможность работы приложения в офлайн-режиме и эффективное управление данными через единый репозиторий.

Четвёртая работа завершила цикл разработки, добавив в приложение расширенные возможности пользовательского интерфейса и фоновой

обработки. Были внедрены компоненты Material Design, реализована интерактивность (например, возможность «лайкать» сообщения), а также настроена фоновая синхронизация данных через WorkManager с уведомлениями о успешном обновлении.

Таким образом для успешного выполнения работы было поставлено четыре задачи:

- 1) Разработать простое Android-приложение "Мессенджер" с использованием архитектуры на основе Activity и Fragment, а также встроенной навигации;
- 2) Реализовать управление состоянием приложения и архитектуру MVVM для хранения и обновления пользовательских данных в экранах;
- 3) Добавить загрузку списка сообщений из API и реализовать сохранение данных в локальную базу Room для офлайн-доступа;
- 4) Улучшить пользовательский интерфейс, добавить уведомления и фоновую синхронизацию сообщений.

В результате выполнения всех задач было создано полноценное Android-приложение, соответствующее современным стандартам разработки. Оно объединяет в себе чистую архитектуру MVVM, работу с сетью и базой данных, интерактивный интерфейс и фоновые процессы, что делает его готовым к дальнейшему расширению и использованию в реальных условиях.

1 Структура проекта

Проект построен в виде модульного приложения для Android, реализующего функционал мессенджера с использованием современного стека технологий и архитектурных подходов. Основная кодовая база расположена в пакете `com.example.mymessenger` и организована в соответствии с принципами чистой архитектуры и паттерном MVVM (Model-View-ViewModel), что обеспечивает чёткое разделение ответственности между компонентами, упрощает тестирование и поддержку кода, а также облегчает дальнейшее расширение функционала (рисунок 1).

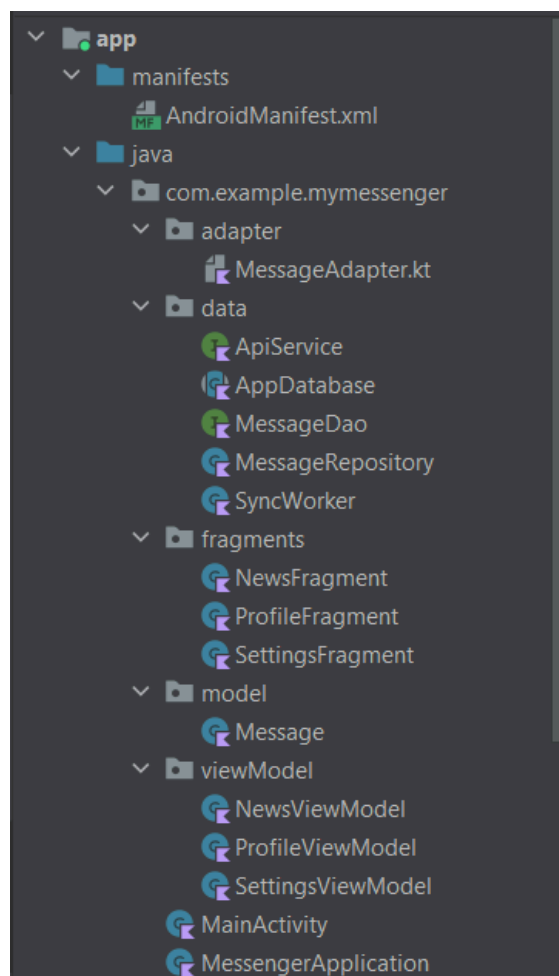


Рисунок 1 – Структура проекта Android Studio

В основе проекта лежит классическое разделение на слои: представление, бизнес-логика и данные. В корне находится файл `AndroidManifest.xml`, который определяет структуру приложения, объявляет активности, разрешения и другие системные компоненты. Главная точка

входа в интерфейс — `MainActivity`, которая отвечает за навигацию между экранами через `BottomNavigationView` и `NavController`. Внутри активности управление передаётся фрагментам, каждый из которых представляет отдельный экран приложения: `NewsFragment` для ленты сообщений, `ProfileFragment` для отображения и редактирования профиля пользователя и `SettingsFragment` для настроек, таких как переключение темы. Все фрагменты спроектированы как реактивные компоненты, которые получают данные от соответствующих `ViewModel` и обновляют интерфейс в ответ на изменения состояния.

Слой `ViewModel`, расположенный в пакете `viewModel`, служит мостом между пользовательским интерфейсом и бизнес-логикой. `NewsViewModel` управляет загрузкой и отображением списка сообщений, взаимодействуя с репозиторием данных. `ProfileViewModel` хранит состояние профиля пользователя — имя и статус, обеспечивая их сохранение при повороте экрана. `SettingsViewModel` отвечает за настройки приложения, включая выбор темы. Все `ViewModel` используют `LiveData` или `StateFlow` для передачи изменений во фрагменты, что гарантирует реактивное обновление интерфейса без потери данных.

Центральным звеном в работе с данными выступает `MessageRepository`, расположенный в пакете `data`. Этот репозиторий объединяет доступ к сетевым источникам и локальному хранилищу, обеспечивая единую точку взаимодействия для `ViewModel`. При наличии сети репозиторий загружает сообщения через `ApiService`, реализованный с помощью `Retrofit` и `Kotlin Coroutines`, и сохраняет их в локальную базу данных `Room` через `MessageDao`. В случае отсутствия подключения данные загружаются из базы, что позволяет приложению работать в офлайн-режиме. База данных инициализируется в классе `AppDatabase`, который настраивает сущности и DAO. Для фоновой синхронизации данных используется `SyncWorker` на основе `WorkManager`, который периодически обновляет список сообщений и показывает уведомления при успешном получении новых данных.

Модели данных, такие как класс `Message`, определены в пакете `model` и используются как для сетевых запросов, так и для сохранения в базе данных. Адаптер `MessageAdapter`, находящийся в пакете `adapter`, отвечает за отображение списка сообщений в `RecyclerView`, поддерживая кастомные элементы интерфейса, включая аватарки, имена, текст сообщений и интерактивные функции, такие как «лайки».

Такая структура не только соответствует лучшим практикам Android-разработки, но и создаёт устойчивую основу для масштабирования, позволяя легко добавлять новые функции, экраны или источники данных без переписывания существующего кода.

2 Создание скелета приложения с помощью Activity и Fragment

Было создано новое Android-приложение с использованием шаблона Empty Activity. В файле `activity_main.xml` реализован основной макет (рисунок 2), содержащий:

- `NavHostFragment` — контейнер для отображения фрагментов,
- `BottomNavigationView` — нижняя панель навигации с тремя пунктами: "Лента", "Профиль", "Настройки".

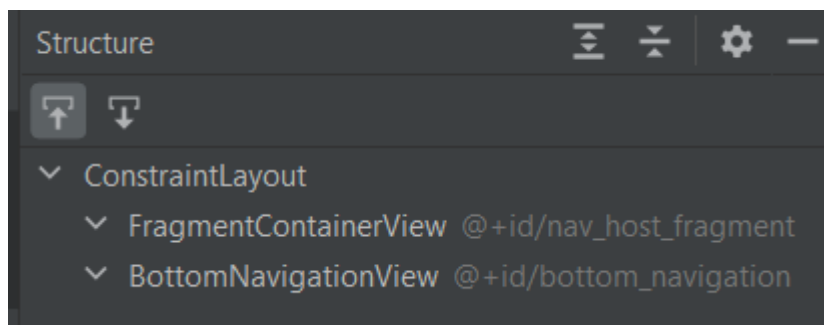


Рисунок 2 – Содержание `activity_main.xml`

В классе `MainActivity` (рисунок 3) настроена связь между `BottomNavigationView` и `NavController`:

```
class MainActivity : AppCompatActivity() {  
  
    private lateinit var binding: ActivityMainBinding  
    private val settingsViewModel: SettingsViewModel by viewModels()  
  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
  
        binding = ActivityMainBinding.inflate(layoutInflater)  
        setContentView(binding.root)  
  
        android.util.Log.d(tag: "Lifecycle", msg: "com.example.mymessenger.MainActivity created")  
  
        settingsViewModel.init(applicationContext)  
  
        val navHostFragment = supportFragmentManager.findFragmentById(R.id.nav_host_fragment) as NavHostFragment  
        val navController = navHostFragment.navController  
  
        binding.bottomNavigation.setupWithNavController(navController)  
  
        if (savedInstanceState == null) {  
            binding.bottomNavigation.selectedItemId = R.id.newsFragment  
        }  
    }  
  
    override fun onDestroy() {  
        super.onDestroy()  
        android.util.Log.d(tag: "Lifecycle", msg: "com.example.mymessenger.MainActivity destroyed")  
    }  
}
```

Рисунок 3 – `MainActivity.kt`

В ресурсах проекта создан файл `nav_graph.xml`, определяющий структуру навигации приложения (рисунок 4). Навигационный граф связан с `MainActivity` через атрибут `app:navGraph` в `NavHostFragment`.

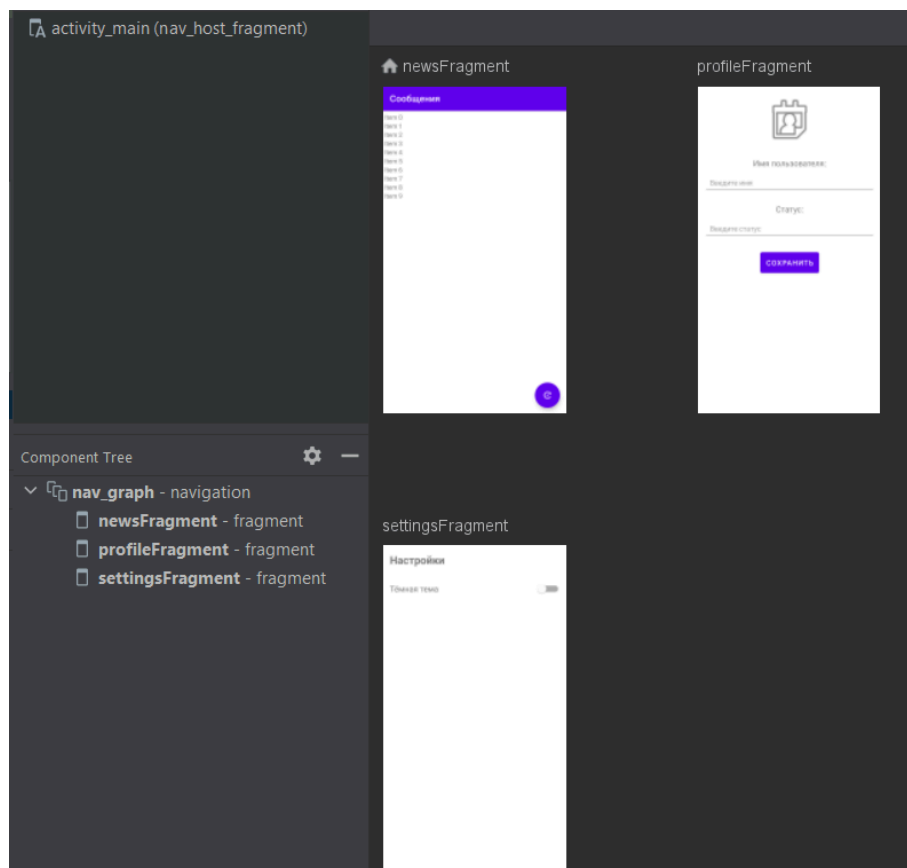


Рисунок 4 – `nav_graph.xml`

В `activity_main.xml` добавлен `BottomNavigationView` с тремя элементами меню, как показано на листинге 1:

Листинг 1 – `activity_main.xml`

```
<com.google.android.material.bottomnavigation.BottomNavigationView
    android:id="@+id/bottom_navigation"
    android:layout_width="0dp"
    android:layout_height="wrap_content"
    app:layout_constraintBottom_toBottomOf="parent"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintStart_toStartOf="parent"
    app:menu="@menu/bottom_nav_menu" />
```

Файл `bottom_nav_menu.xml` представлен на листинге 2. Он определяет иконки и заголовки пунктов:

Листинг 2 – bottom_nav_menu.xml

```
<?xml version="1.0" encoding="utf-8"?>
<menu xmlns:android="http://schemas.android.com/apk/res/android">
    <item
        android:id="@+id/newsFragment"
        android:icon="@android:drawable/ic_dialog_email"
        android:title="Лента" />

    <item
        android:id="@+id/profileFragment"
        android:icon="@android:drawable/ic_menu_my_calendar"
        android:title="Профиль" />

    <item
        android:id="@+id/settingsFragment"
        android:icon="@android:drawable/ic_menu_preferences"
        android:title="Настройки" />
</menu>
```

2.1 Фрагмент ленты (NewsFragment)

Создан NewsFragment с базовым макетом, который впоследствии будет служить основной страницей приложения со списком полученных сообщений. На данном этапе он содержит заглушку, как показано на рисунке 5.

```
class NewsFragment : Fragment() {

    private var _binding: FragmentNewsBinding? = null
    private val binding get() = _binding!!

    override fun onCreateView(
        inflater: LayoutInflater, container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View {
        _binding = FragmentNewsBinding.inflate(inflater, container, false)
        Log.d("Lifecycle", "com.example.mymessenger.NewsFragment onCreateView")
        return binding.root
    }

    override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
        super.onViewCreated(view, savedInstanceState)
        // Заглушка для новостной ленты
        binding.textNews.text = "Здесь будет лента новостей"
    }
}
```

Рисунок 5 – Базовый NewsFragment.kt

2.2 Фрагмент профиля (ProfileFragment)

Экран Профиля пользователя содержит два поля для ввода имени и статуса, поля отражения актуальных данных и аватарке пользователя (рисунок 6).

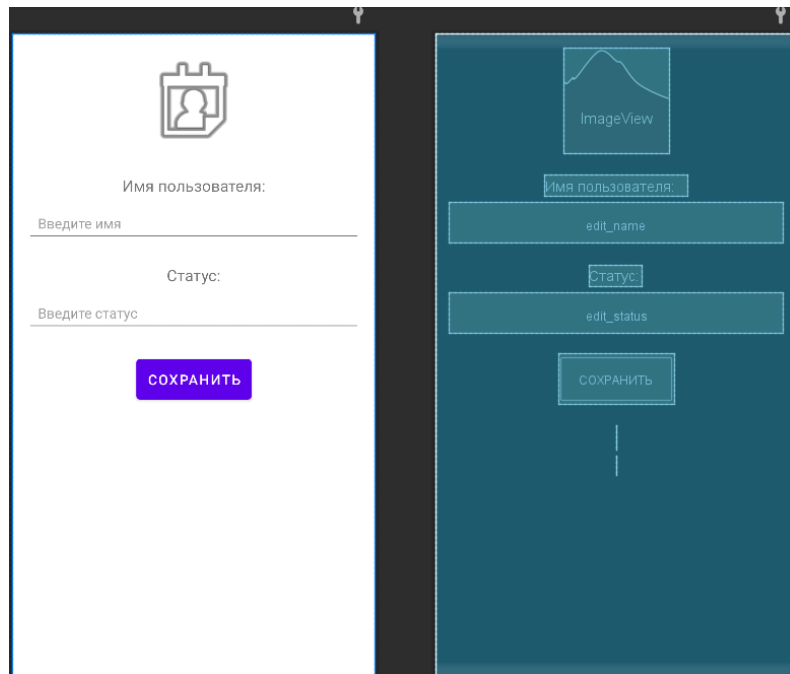


Рисунок 6 – Макет страницы Профиля

2.3 Фрагмент настроек (SettingsFragment)

Создан экран настроек с переключателем темы, который, как и остальные фрагменты, привязан к ViewModel для изменения и сохранения состояния (листинг 3).

Листинг 3 – Изменение темы в SettingsFragment

```
class SettingsFragment : Fragment() {  
  
    private var _binding: FragmentSettingsBinding? = null  
    private val binding get() = _binding!!  
  
    private val settingsViewModel: SettingsViewModel by  
    activityViewModels()  
  
    override fun onCreateView(  
        inflater: LayoutInflater, container: ViewGroup?,  
        savedInstanceState: Bundle?  
    ): View {  
        _binding = FragmentSettingsBinding.inflate(inflater,  
            container, false)  
        Log.d("Lifecycle",  
            "com.example.mymessenger.fragments.SettingsFragment onCreateView")  
    }  
}
```

```

        return binding.root
    }

    override fun onViewCreated(view: View, savedInstanceState:
Bundle?) {
        super.onViewCreated(view, savedInstanceState)

        binding.themeSwitch.isChecked =
settingsViewModel.isDarkTheme()

        binding.themeSwitch.setOnCheckedChangeListener { _, isChecked
->
            settingsViewModel.setDarkTheme(isChecked)
        }
    }
}

```

Каждый элемент управления темой был связан с соответствующим методом ViewModel через слушатели событий. Переключатель использует OnCheckedChangeListener, который вызывает метод setTheme() при изменении своего состояния. Кнопки управления используют стандартные OnClickListener для вызова соответствующих методов ViewModel.

3 Внедрение ViewModel

3.1 ProfileViewModel

В рамках данного этапа был спроектирован и реализован класс ProfileViewModel (рисунок 7), который является ключевым компонентом для управления состоянием профиля пользователя. Этот класс наследуется от стандартного ViewModel из Android Architecture Components, что обеспечивает его корректный жизненный цикл и сохранение данных при изменениях конфигурации устройства.

```
class ProfileViewModel : ViewModel() {

    private val _userName = MutableLiveData(value: "vae eva")
    private val _userStatus = MutableLiveData(value: "live laugh love")

    val userName: LiveData<String> = _userName
    val userStatus: LiveData<String> = _userStatus

    init {
        Log.d(tag: "ViewModelLifecycle", msg: "ProfileViewModel создан")
    }

    fun updateUser(newName: String) {
        Log.d(tag: "ProfileViewModel", msg: "Имя обновлено: $newName")
        _userName.value = newName
    }

    fun updateUserStatus(newStatus: String) {
        Log.d(tag: "ProfileViewModel", msg: "Статус обновлен: $newStatus")
        _userStatus.value = newStatus
    }

    override fun onCleared() {
        super.onCleared()
        Log.d(tag: "ViewModelLifecycle", msg: "ProfileViewModel уничтожен")
    }
}
```

Рисунок 7 – Класс ProfileViewModel

Основной задачей ProfileViewModel стало хранение и управление двумя основными элементами профиля пользователя: именем и статусом. Для реализации этой функциональности были использованы объекты MutableLiveData<String>, которые предоставляют реактивный способ обновления данных.

Важной особенностью реализации является использование приватных `MutableLiveData` объектов с публичными неизменяемыми `LivData` представлениями. Этот паттерн, известный как "защитное копирование" или "immutable exposure", предотвращает неконтролируемое изменение состояния извне `ViewModel`, обеспечивая тем самым целостность данных и соблюдение принципа единой ответственности.

3.2 Изменение `ProfileFragment`

Логика работы `ProfileFragment` была переработана для интеграции с `ProfileViewModel`. Ключевым аспектом интеграции стала настройка наблюдателей (`Observer`) для `LivData` объектов `ViewModel`. Для каждого `LivData` (имя и статус) был создан отдельный наблюдатель, который обновляет UI при изменении соответствующих данных. Использование `viewLifecycleOwner` в качестве владельца жизненного цикла наблюдателей гарантирует, что наблюдатели будут автоматически удаляться при уничтожении View фрагмента, предотвращая тем самым утечки памяти.

Пример применения такого подхода показан в листинге 4 на изменении имени пользователя.

Листинг 4 – Прослушивание изменения имени

```
viewModel.userName.observe(viewLifecycleOwner) { name ->
    binding.tvCurrentName.text = "Имя: $name"
    binding.editName.setText(name) }

binding.btnSave.setOnClickListener {
    val newName = binding.editName.text.toString()
    val newStatus = binding.editStatus.text.toString()

    viewModel.updateUserName(newName)
    viewModel.updateUserStatus(newStatus) }

binding.editName.setOnFocusChangeListener { _, hasFocus ->
    if (!hasFocus) {
        viewModel.updateUserName(binding.editName.text.toString()) } }
```

3.3 SettingsViewModel

SettingsViewModel, отвечающий за управление настройками визуального оформления приложения, хранит состояние текущей темы в виде булевого значения, где true соответствует тёмной теме, а false — светлой. Использование LiveData<Boolean> для хранения состояния темы позволяет всем заинтересованным компонентам UI автоматически обновляться при изменении этого состояния.

Класс предоставляет три основных метода для управления темой: toggleTheme() для переключения между темами, setTheme(isDark: Boolean) для явной установки определённой темы и стандартный метод onCleared() для очистки ресурсов.

Листинг 5 – Смена темы в SettingsViewModel

```
private fun applyTheme(isDark: Boolean, saveToPrefs: Boolean =
true) {
    val mode = if (isDark) {AppCompatDelegate.MODE_NIGHT_YES
    } else {AppCompatDelegate.MODE_NIGHT_NO}

    viewModelScope.launch {
        AppCompatDelegate.setDefaultNightMode(mode)
        if (saveToPrefs) {
            sharedPreferences.edit().putBoolean(KEY_IS_DARK_THEME,
isDark).apply()
        }
    }
}

fun setDarkTheme(isDark: Boolean) {
    applyTheme(isDark)
    Log.d("SettingsViewModel", "Тема установлена: ${if (isDark)
"темная" else "светлая"}")
}

fun isDarkTheme(): Boolean {
    return sharedPreferences.getBoolean(KEY_IS_DARK_THEME, false)
}
```

4 Загрузка данных из API

Первым шагом в реализации функционала загрузки сообщений стало проектирование модели данных, которая будет использоваться на всех уровнях приложения — от сетевых запросов до локального хранения и отображения в пользовательском интерфейсе. Был создан класс `Message` (рисунок 8), представляющий собой data-класс Kotlin с аннотациями для совместимости с библиотекой `Room` и `Gson` для сериализации/десериализации.

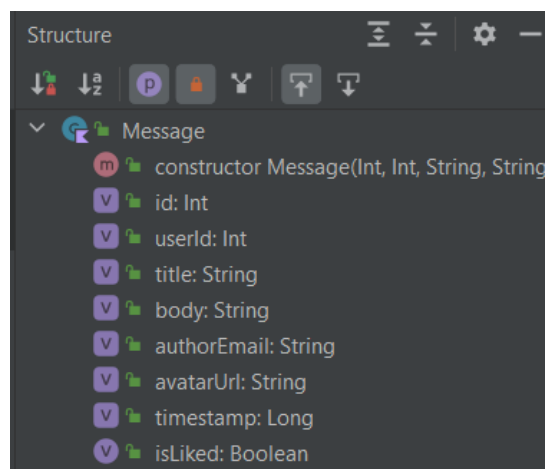


Рисунок 8 – Структура класса Сообщение

Ключевой особенностью модели стало использование единого класса для всех слоёв приложения, что упрощает преобразование данных между форматами. Аннотация `@Entity` указывает `Room`, что этот класс представляет таблицу в базе данных, а `@SerializedName` обеспечивает корректное маппинг полей при парсинге JSON ответов от API.

Для организации сетевых запросов была реализована сервисная прослойка на основе библиотеки `Retrofit 2`, которая предоставляет декларативный способ описания API endpoints. Основой стал интерфейс `ApiService` (рисунок 9), содержащий определение метода для взаимодействия с сервером.


```

interface ApiService {

    @GET("posts")
    suspend fun getMessages(): Response<List<Message>>

    companion object {
        private const val BASE_URL = "https://jsonplaceholder.typicode.com/"

        fun create(): ApiService {
            val retrofit = Retrofit.Builder()
                .baseUrl(BASE_URL)
                .addConverterFactory(GsonConverterFactory.create())
                .build()

            return retrofit.create(ApiService::class.java)
        }
    }
}

```

Рисунок 9 – ApiService-интерфейс

Эндпоинтом API-запроса стал открытый сайт “{JSON} Placeholder”, содержащий “Free fake and reliable API for testing and prototyping”.

Для взаимодействия с локальной базой данных был разработан интерфейс MessageDao, который определяет все необходимые операции с таблицей сообщений. Использование аннотаций Room позволяет компилятору генерировать реализацию этих методов, гарантируя корректность SQL-запросов и типобезопасность (рисунок 10).

```

@Dao
interface MessageDao {

    @Query("SELECT * FROM messages ORDER BY timestamp DESC")
    fun getAllMessages(): Flow<List<Message>>

    @Query("SELECT * FROM messages ORDER BY timestamp DESC")
    suspend fun getAllMessagesSync(): List<Message>

    @Transaction
    suspend fun upsertAll(messages: List<Message>) {
        messages.forEach { message ->
            val existing = getMessageById(message.id)
            if (existing == null) {
                insertMessage(message)
            } else {
                val updatedMessage = message.copy(isLiked = existing.isLiked)
                updateMessage(updatedMessage)
            }
        }
    }

    @Query("SELECT * FROM messages WHERE id = :messageId")
    suspend fun getMessageById(messageId: Int): Message?

    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun insertMessage(message: Message)

    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun insertAll(messages: List<Message>)

```

Рисунок 10 – Интерфейс MessageDao

Центральным компонентом локального хранилища стал класс AppDatabase, расширяющий RoomDatabase и служащий точкой входа для работы с базой данных. Этот класс реализован как синглтон для обеспечения единственного экземпляра базы данных в рамках всего приложения.

Репозиторий MessageRepository был спроектирован как центральный компонент, который инкапсулирует всю логику работы с данными, скрывая от ViewModel детали реализации источников данных (рисунок 11). Основной задачей репозитория стало определение, откуда загружать данные — из сети или из локальной базы, в зависимости от доступности подключения.

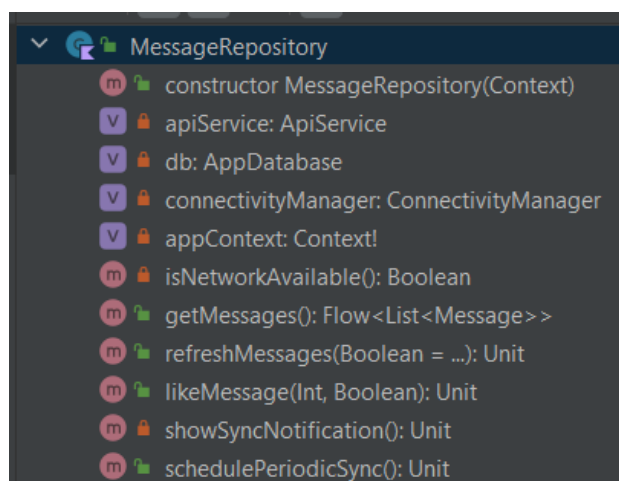


Рисунок 11 – Структура MessageRepository

Репозиторий включает комплексную систему обработки ошибок, которая гарантирует, что приложение останется функциональным даже при возникновении проблем с сетью или базой данных. В случае ошибки при загрузке из сети репозиторий автоматически переключается на локальную базу, обеспечивая бесшовный опыт использования приложения.

Отдельное внимание было уделено обработке операции "лайкания" сообщений. Реализация обеспечивает мгновенную обратную связь пользователю (изменение состояния в UI) с последующей асинхронной синхронизацией с сервером при наличии сети.

5 Отображение списка на странице Новостей

Для отображения списка сообщений был разработан кастомный адаптер MessagesAdapter, который расширяет RecyclerView.Adapter и обеспечивает эффективное отображение элементов списка с поддержкой различных типов взаимодействия, включая обработку кликов на лайки (рисунок 12).

```
class MessagesAdapter(  
    private val onItemClick: (Message) -> Unit = {},  
    private val onLikeClick: (Message) -> Unit = {}  
) : ListAdapter<Message, MessagesAdapter.MessageViewHolder>(MessageDiffCallback()) {  
  
    class MessageViewHolder(itemView: View) : RecyclerView.ViewHolder(itemView) {  
        private val tvAuthorEmail: TextView = itemView.findViewById(R.id.tv_author_email)  
        private val tvTitle: TextView = itemView.findViewById(R.id.tv_message_title)  
        private val ivAvatar: ImageView = itemView.findViewById(R.id.iv_avatar)  
        private val btnLike: ImageButton = itemView.findViewById(R.id.btn_like)  
  
        fun bind(message: Message, onItemClick: (Message) -> Unit, onLikeClick: (Message) -> Unit) {  
            tvTitle.text = message.title  
            tvAuthorEmail.text = message.authorEmail  
  
            val avatarUrl = message.avatarUrl  
  
            if (message.avatarUrl.isNotEmpty()) {  
                Glide.with(itemView.context).requestManager  
                    .load(avatarUrl).requestBuilder<Drawable?>  
                    .transform(CircleCrop())  
                    .placeholder(android.R.drawable.ic_menu_gallery)  
                    .error(android.R.drawable.ic_dialog_alert)  
                    .into(ivAvatar)  
            }  
  
            btnLike.setImageResource(  
                if (message.isLiked) android.R.drawable.star_big_on  
                else android.R.drawable.star_big_off  
            )  
        }  
    }  
}
```

Рисунок 11 – MessagesAdapter

Адаптер использует паттерн ViewHolder для оптимизации производительности списка и включает поддержку загрузки изображений через библиотеку Glide.

Для каждого элемента списка сообщений был создан макет item_message.xml, который реализует современный дизайн в соответствии с принципами Material Design. Макет включает карточку (CardView) с закругленными углами и тенями для создания визуальной глубины. Дизайн макета обеспечивает четкую визуальную иерархию информации: аватар

автора выделяет начало сообщения, текст сообщения занимает центральное место, а панель действий с лайками располагается в нижней части карточки.

NewsViewModel был спроектирован как посредник между UI (NewsFragment) и репозиторием данных. Основная задача ViewModel — управление состоянием экрана ленты сообщений, включая загрузку данных, обработку ошибок и управление состоянием обновления.

Листинг 6 – Методы загрузки сообщений

```
@RequiresApi (Build.VERSION_CODES.M)
fun loadMessages() {
    viewModelScope.launch {
        _isLoading.value = true
        _error.value = null

        try {
            repository.getMessages().collect { messagesList ->
                _isLoading.value = false
                Log.d("NewsViewModel", "Загружено
                ${messagesList.size} сообщений")}
        } catch (e: Exception) {
            _error.value = "Ошибка загрузки: ${e.message}"
            _isLoading.value = false
            Log.e("NewsViewModel", "Ошибка: ${e.message}")
        }
    }

    @RequiresApi (Build.VERSION_CODES.M)
    fun refreshMessages() {
        _isRefreshing.value = true
        viewModelScope.launch {
            try {
                repository.refreshMessages(showNotification = true)
            } catch (e: Exception) {
                Log.e("NewsViewModel", "Ошибка обновления:
                ${e.message}")
            } finally {
                _isRefreshing.value = false
            }
        }
    }
}
```

Окончательная версия NewsFragment была существенно переработана для интеграции всех реализованных компонентов: ViewModel, адаптера RecyclerView и поддержки различных состояний экрана. Пример доработки отмечен на рисунке 12.

```

@RequiresApi(Build.VERSION_CODES.M)
override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
    super.onViewCreated(view, savedInstanceState)

    setupRecyclerView()
    setupObservers()
    setupFloatingActionButton()
    requestPermissions()

    viewModel.schedulePeriodicSync()
}

@RequiresApi(Build.VERSION_CODES.M)
private fun setupRecyclerView() {
    messagesAdapter = MessagesAdapter(
        onItemClick = { message ->
            Snackbar.make(binding.root, text: "Сообщение: ${message.title}", Snackbar.LENGTH_SHORT).show()
        },
        onLikeClick = { message ->
            viewModel.likeMessage(message.id, !message.isLiked)
        }
    )

    binding.recyclerViewMessages.apply { this: RecyclerView
        layoutManager = LinearLayoutManager(requireContext())
        adapter = this@NewsFragment.messagesAdapter
    }
}

```

Рисунок 12 – Методы NewsFragment

Макет fragment_news.xml был спроектирован для отображения различных состояний экрана. Используются современные компоненты Material Design, такие как SwipeRefreshLayout и FloatingActionButton.

Результат работы над внешним видом страницы Сообщений представлен на рисунке 13.

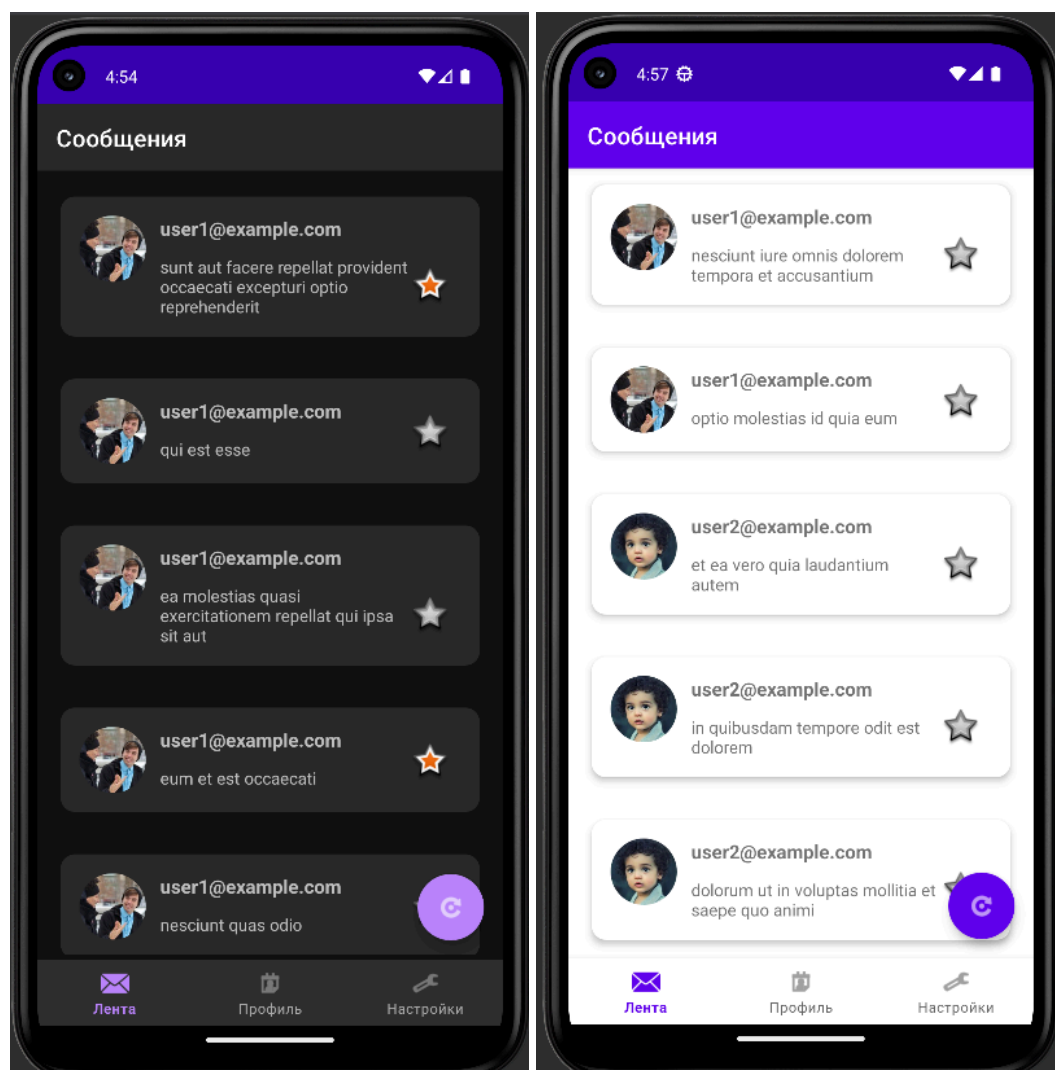


Рисунок 12 – Новостная лента

6 Улучшение пользовательского интерфейса

Класс `SyncWorker` реализован как наследник `CoroutineWorker` — специализированного компонента библиотеки `Android WorkManager`, предназначенного для выполнения отложенных и периодических задач. Метод `doWork()` является точкой входа для выполнения фоновой задачи и реализован с учетом современных практик Android-разработки. Основная задача `SyncWorker` — выполнение периодической синхронизации сообщений без активного участия пользователя (листинг 7). Это реализуется через:

- 1) загрузку новых сообщений с сервера при наличии сетевого подключения;
- 2) синхронизацию статусов сообщений (прочитано, доставлено, лайки);
- 3) обновление локальной базы данных для обеспечения офлайн-доступа;
- 4) обработку конфликтов данных при расхождениях между локальной и серверной версиями.

Листинг 7 – Периодическое обновление списка сообщений

```
fun schedulePeriodicSync() {
    val constraints = Constraints.Builder()
        .setRequiredNetworkType(NetworkType.CONNECTED)
        .setRequiresBatteryNotLow(true)
        .build()

    val syncRequest = PeriodicWorkRequestBuilder<SyncWorker>(
        15, TimeUnit.MINUTES,
        5, TimeUnit.MINUTES
    )
        .setConstraints(constraints)
        .addTag("message_sync")
        .build()

    WorkManager.getInstance(appContext)
        .enqueueUniquePeriodicWork(
            "message_sync_work",
            ExistingPeriodicWorkPolicy.KEEP,
            syncRequest
        )
}
```


В результате успешного обновления списка сообщений пользователю приходит фоновое уведомление, как показано на рисунке 13.

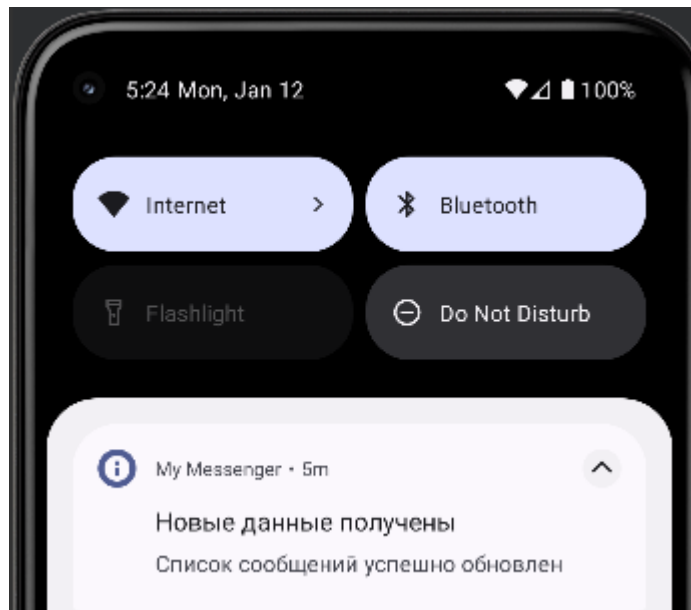


Рисунок 13 – Уведомление об обновлении

Уведомление построено с помощью `NotificationCompat.Builder` и представлено на листинге 8.

Листинг 8 – Построение уведомления

```
val notification = NotificationCompat.Builder(appContext,
"sync_channel")
    .setSmallIcon(android.R.drawable.ic_dialog_info)
    .setContentTitle("Новые данные получены")
    .setContentText("Список сообщений успешно обновлен")
    .setPriority(NotificationCompat.PRIORITY_DEFAULT)
    .setAutoCancel(true)
    .build()

notificationManager.notify(1, notification)
```

ЗАКЛЮЧЕНИЕ

В результате выполнения курсовой работы было успешно разработано и реализовано полнофункциональное Android-приложение «Мессенджер», соответствующее современным стандартам мобильной разработки. В ходе поэтапного выполнения четырёх лабораторных работ была создана архитектурно-грамотная, масштабируемая и отказоустойчивая система, объединяющая пользовательский интерфейс, бизнес-логику и работу с данными. Приложение демонстрирует комплексное применение принципов реактивного программирования и паттерна MVVM, что обеспечивает чёткое разделение ответственности между слоями и упрощает дальнейшую поддержку и расширение функционала.

Ключевым достижением работы стало построение многоуровневой архитектуры, включающей сетевое взаимодействие через Retrofit, локальное хранение данных с использованием Room, фоновую синхронизацию через WorkManager и интерактивный пользовательский интерфейс на основе Material Design. Реализация репозитория как единой точки доступа к данным позволила обеспечить бесшовный переход между онлайн- и офлайн-режимами работы, что критически важно для мессенджера как класса приложений. Система уведомлений и разрешений была интегрирована с соблюдением рекомендаций платформы, гарантируя корректную работу в фоне и уважение к пользовательским настройкам конфиденциальности.

Особое внимание было уделено созданию отзывчивого и интуитивно понятного интерфейса. Интеграция системы «лайков» с оптимистичными обновлениями и фоновой синхронизацией демонстрирует понимание современных подходов к построению реактивных интерфейсов. Использование компонентов Material Design, включая FloatingActionButton с расширяемым меню и BottomNavigationView, обеспечило соответствие приложения актуальным дизайн-стандартам. Механизм уведомлений о новых сообщениях, интегрированный с системой синхронизации, обеспечивает

своевременное информирование пользователя без негативного влияния на автономность устройства.

В целом, выполненная работа подтвердила уровень освоения технологий Android-разработки и способность к комплексному проектированию программных систем. Полученное приложение готово к использованию и служит надёжной основой для дальнейшего развития — добавления новых функций, таких как голосовые и видеозвонки, групповые чаты или интеграция с дополнительными сервисами. Практическая значимость работы заключается в демонстрации профессионального подхода к созданию мобильных приложений, сочетающего техническую грамотность, внимание к пользовательскому опыту и соблюдение лучших практик индустрии.